

Which Parts of the “Modern Transformer Recipe” Actually Matter?

A Controlled Ablation on TinyStories

Cohen Brunner Thomas

June 2026

Abstract

These days when people train language models they use a big list of little tricks. The tricks I look at are Muon, QK-norm, learnable QK-gain, LayerScale, value residuals, z-loss, and logit softcapping. People throw them all together and call it the “modern recipe.” I wanted to know which ones actually do anything. So I tested them one at a time and also in groups. I used the same transformer model every time. It had about 97.6M parameters and it was a decoder-only model. Every run got the same amount of training. That was 30.72M tokens and 1000 steps. I ran 22 different setups and I did each one 3 times with 3 different seeds. That makes 66 runs in total. I did not test every possible combination because that would be way too many. I picked the important ones instead. Two tricks did most of the work. Swapping the AdamW optimizer for Muon made the final validation loss go down by about 0.13 nats. Adding QK-norm also made it go down by about 0.13 nats. The cool part is that these two work way better together than apart. Together they give you about 0.24 nats which is much more than just adding them up. LayerScale and value residuals each help a little bit too at about 0.05 nats and they also help each other a little. QK-gain and logit softcap basically do nothing ($|\Delta| < 0.002$ nats). z-loss actually makes things a tiny bit worse at this size (+0.016 nats). The best single setup was not the one with everything turned on. It was the full Muon recipe but with z-loss taken out (1.651 val loss). The everything-on setup got 1.669. All 66 runs finished and none of them broke.

1 Introduction

When people share how they train big language models they keep piling tricks on top of each other. Each new trick gets added to the old ones and then they test the whole pile at once. The problem with this is you never really know which trick is doing the work. You also do not know which ones are useless and which ones are even making things worse for the size of model you are using. It also makes the recipe hard to copy. If you copy the whole pile you get the good parts but you also get all the dead weight and all the ways it can go wrong.

In this paper I take the seven toggles in the `TinyForge` training stack and I test each one by itself and also in groups. I keep everything else exactly the same the whole time. That means the model size, the data, the amount of training, the learning-rate schedule, the weight decay, the gradient clipping, and the order of the batches all stay fixed. My question is pretty simple. With the same amount of compute, what does each trick add to the validation loss, and do the tricks change how each other behave?

2 Related work

The closest paper to mine is by Narang et al. [9]. They ran a big test over lots of proposed Transformer changes in a shared setting. They found that most of the changes do not actually

carry over. A lot of the time the gains came from the original code and not the idea itself, and not many changes helped on every task. I want the same kind of thing they do, which is testing recipe ingredients properly instead of just trusting that they work. But I am looking at something different. They tested lots of architectures and tasks at one size to see which changes work everywhere. I keep one model, one corpus, and one token budget and I look at how the tricks inside one modern stack work with each other instead. The main thing I find is that most of the benefit comes from one Muon and QK-norm pair and not from the seven toggles spread out. The tricks I test are all pretty new and they came out after that survey. They are Muon [7], QK-norm [5], LayerScale [12], value residuals [14], and the z-loss and logit-softcap stabilizers from PaLM [1] and Gemma 2 [4]. For the design I use a fractional factorial setup with matched on/off pairs and some special 2×2 interaction cells. This lets me measure both the main effects and the interactions with only a small number of runs. A simple one-at-a-time sweep would miss those interactions.

This kind of small-budget single-GPU work follows a line of other small-scale language-model papers. The closest ones are Cramming [3], which looked at the whole pretraining pipeline on a one-day single-GPU budget, and the TinyStories corpus [2], which is simple made-up text that lets you train a model of about 10^2 M parameters until it makes sense in a short time. That setting is the reason I can afford to do 66 runs. But it also limits what I can say. It is exactly the short and small-vocab setting where stability tricks like z-loss are supposed to do nothing anyway. I come back to that point later in the discussion.

3 The components under study

All seven are toggles that sit on top of one shared backbone (12 layers, 12 heads, $d_{\text{model}} = 768$, SwiGLU MLP [10], RMSNorm pre-norm [13], RoPE [11], untied LM head). When you turn all the toggles off you get the AdamW baseline [8].

Toggle	What it does	Where
optimizer	Muon [7] (Newton-Schulz-orthogonalized momentum, <code>match_rms_adamw</code> LR) on 2D body matrices plus AdamW on embeddings/head/scales, vs. pure AdamW	optimizer
qk_norm	Per-head RMSNorm on Q and K before RoPE [5]	attention
qk_gain	Learnable per-head multiplicative gain folded into Q (scales scores pre-softmax)	attention
layerscale	Learnable per-channel residual gates on the attention and MLP branches (CaiT [12]), init 0.1	residual
value_residual	Each layer mixes its value with layer-0's value via a learnable per-layer λ (ResFormer [14])	attention
z_loss	<code>lse_square_scale</code> = $1e-4$ regularizer on the log-partition [1]	loss
softcap	tanh logit softcap at 30.0	loss

z-loss and softcap are loss-side options of `LigerFusedLinearCrossEntropyLoss` [6]. The rest are changes to the model or the optimizer. The way I split Muon and AdamW and the weight-decay rule both match the production `simple.py` stack.

4 Experimental setup

Model. About 97.6M parameters. The number changes a tiny bit between setups (from 97,536,768 to 97,573,787) because some toggles add a few extra scale or gate parameters. That is less than 0.04% so it does not mess up the comparison.

Data. roneneldan/TinyStories [2]. I tokenized it once with a fixed 8192-token byte-level BPE into a memory-mapped `uint16` corpus. Every run reads the exact same (x, y) blocks in the exact same order. There is no shuffling and the batches are deterministic. So the only things that change between runs are the toggles and the seed.

Budget. 1000 optimizer steps, batch size 15, gradient accumulation 1, block size 2048. That works out to 30,720 tokens per step which is **30.72M tokens per run**. The LR peak is $1e-3$ with a 99-step linear warmup and then cosine decay down to $0.1\times$ the peak. Weight decay is 0.1, gradient-norm clip is 1.0, and I use bf16 autocast with `torch.compile`. The validation loss is the mean cross-entropy over 100 held-out blocks at the final step.

Design. I did not run the full 2^7 grid because that is 128 setups. Instead I used a fractional factorial so the budget goes to the questions I care about:

- **Anchors.** All-off (pure AdamW baseline) and all-on (full Muon recipe).
- **Add-one.** Baseline plus exactly one toggle, to get clean main effects for the features I expect to matter on their own.
- **Leave-one-out.** The full recipe with exactly one toggle taken out, to see what each one adds inside the whole stack.
- **Targeted 2×2 cells.** Muon $\times$ QK-norm, LayerScale $\times$ value-residual, z-loss $\times$ softcap, z-loss $\times$ QK-gain, and softcap $\times$ QK-gain, for the interaction terms.

That is $22 \text{ unique setups} \times 3 \text{ seeds} = 66 \text{ runs}$. I work out the main effects over matched on/off pairs (setups that are the same except for the one factor). The interactions come from matched 2×2 quadruples. I report effects as $\Delta(\text{final val loss}) = \text{ON} - \text{OFF}$. So a **negative number means the trick helps**.

5 Results

5.1 Main effects

Factor	mean Δ	std	n pairs	verdict
qk_norm	-0.128	0.070	9	large help
optimizer (Muon)	-0.128	0.079	9	large help
value_residual	-0.053	0.032	9	moderate help
layerscale	-0.051	0.034	9	moderate help
qk_gain	-0.002	0.008	15	negligible
softcap	-0.000	0.010	15	negligible
z_loss	+0.016	0.008	15	mildly harmful

Two things really jump out. First, QK-norm and the Muon optimizer are doing most of the work in the recipe. They are each worth about 0.13 nats which is about ten times more than the next group. Second, **z-loss keeps hurting** at this size. Its effect is positive on average and the spread is tight (std 0.008 over 15 pairs). So this is a real thing and not just noise. z-loss is meant to keep things numerically stable for big-vocab and long training. But with an 8192-token vocab and only 1000 steps there is no instability to fix in the first place (no runs broke). So the penalty just drags the model away from the cross-entropy goal.

5.2 Interactions

interaction = [effect of A | B on] − [effect of A | B off]. **A negative number means synergy** (A helps more when B is on). A positive number means redundancy or antagonism.

A	B	interaction	reading
optimizer	qk_norm	−0.158	strong synergy
layerscale	value_residual	−0.071	synergy
softcap	qk_gain	−0.007	~none
z_loss	softcap	+0.006	~none
z_loss	qk_gain	+0.003	~none

The Muon×QK-norm interaction is the big one. The 2×2 cell means make it really clear:

	QK-norm off	QK-norm on
AdamW	1.987	1.929
Muon	1.964	1.749

On its own Muon gives about 0.023 nats and QK-norm gives about 0.058 nats. Together they give about 0.238 which is roughly three times more than the two added up. Muon makes the body weight updates more even and QK-norm keeps the size of the query and key projections under control before they go into attention. The numbers say these two work really well as a team. So **most of the “modern recipe” benefit is packed into this one pair**. LayerScale and value residuals do the same kind of thing but weaker. They each gate or re-route the residual stream and they help each other a bit (−0.071).

5.3 Run rankings and the kitchen-sink trap

The best setups are all built on Muon plus QK-norm:

The top three runs (all three seeds) are the full recipe *with z-loss taken out*. They beat the everything-on setup by about 0.018 nats which fits the main-effect sign of z-loss. Taking QK-norm out of the full recipe (rank 22, 1.781) is the most damaging leave-one-out. That backs up QK-norm being the most important piece. Also the full recipe under **AdamW** (1.847) never even reaches the worst Muon recipe run. That shows how much of the gain goes through the optimizer and QK-norm channel.

From start to finish the recipe takes the model from 1.987 (baseline) down to 1.651 (best). That is a 0.336-nat drop and the Muon×QK-norm pair is doing most of it.

rank	configuration	final val loss
1	Muon, all on except z-loss	1.651 (seed mean 1.653)
4	Muon, full recipe (all on)	1.669 (seed mean 1.670)
19	Muon + QK-norm only	1.747
22	Muon, all on except QK-norm	1.781
n/a	AdamW baseline (all off)	1.987
n/a	AdamW full recipe	1.847

5.4 Wall-clock cost

Every run records the total `wall_time_s`. The runs take between 260 and 325 s (mean 278 s, 1000 steps). The per-component cost, measured with the same matched on/off pairs as the loss effects, is all in one place.

Factor	Δs	$\Delta\%$	std (s)	reading
optimizer (Muon)	+19.9	+7.4	0.66	real cost, tight
qk_norm	+8.7	+3.1	7.12	cost within noise
value_residual	+2.5	+0.9	10.09	noise
layerscale	+1.6	+0.6	10.45	noise
softcap	+0.3	+0.1	0.62	negligible
z_loss	−0.2	−0.1	0.50	negligible
qk_gain	−0.7	−0.3	7.35	noise

Swapping AdamW for Muon is the only clear cost. It is about 20 s per run (+7.4%, std 0.66 over 9 pairs) which is the Newton-Schulz orthogonalization overhead on the body matrices. QK-norm looks like +8.7 s but its std (7.1 s) is bigger than its mean. So its little per-step RMSNorm cost basically gets lost in the shared-GPU timing jitter. Everything else is under 1% and you cannot tell it apart from noise. So the cost ranking matches the accuracy ranking. The two pieces that do the work are also the only two that cost anything you can measure. And the price is cheap for what you get. The best setup (298 s) costs +14.6% more wall time than the baseline (260 s) for its −0.336-nat gain. The Muon plus QK-norm pair alone (292 s) buys −0.238 nats for about 12% more time, and the rest of the components add about 0.10 nats more for almost no extra cost.

5.5 Does the verdict flip at larger vocab?

The shakiest results in the main study are the inert/harmful calls on z-loss and softcap. They are both output-softmax stabilizers and they are only supposed to help when the LM-head logits actually blow up, which is at large vocab or long schedules. To test that directly I re-ran the z-loss×softcap 2×2 cell across a vocabulary sweep. I held everything else at a cheaper base setting (8 layers, 8 heads, $d_{\text{model}} = 512$, 1000 steps, block size 1024) and I only changed the vocabulary: 8192 → 16384 → 32768. Each vocab gets its own freshly tokenized corpus. 32768 is the top point because the byte-level BPE maxes out around 26.9k merges on TinyStories. The effects are the same ON−OFF $\Delta(\text{final val loss})$ as before (negative means it helps) and each one is averaged over the two settings of the other factor.

The flip I was expecting does not happen and z-loss goes the wrong way. Instead of becoming useful as the softmax gets bigger, its penalty just keeps getting *more harmful* as the vocab grows

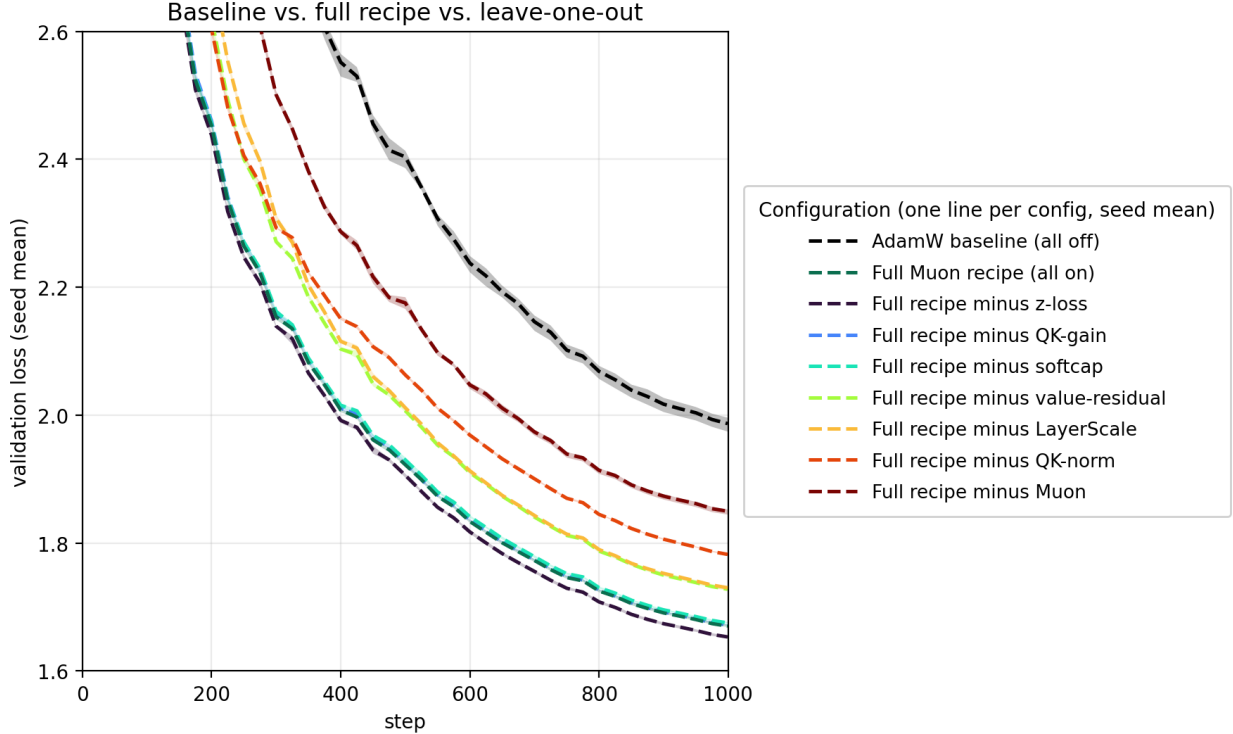


Figure 1: Validation-loss curves with one line per setup (the mean over 3 seeds, with a shaded min-to-max band that is narrow because the seed spread, std about 0.003 to 0.008 nats, is small next to the gaps between setups) for the two anchors (the all-off AdamW baseline in dashed black and the all-on full Muon recipe in green) and the leave-one-out runs (the full recipe with one piece taken out, labelled by the piece that was dropped). The full recipe pulls away from the baseline early and keeps the gap the whole time. Out of the leave-one-outs, dropping Muon or QK-norm hurts by far the most, while dropping z-loss actually *helps* a little over the full recipe.

(+0.007 $\rightarrow$ +0.012 $\rightarrow$ +0.016 nats). It triples over a $4\times$ vocabulary jump. Softcap is the gentler tanh clamp and it is mildly useful at 8k to 16k but it fades to nothing by 32k. The $z\text{-loss} \times \text{softcap}$ interaction stays small and positive the whole way (+0.004 $\rightarrow$ +0.010). So inside the vocabulary range the corpus can handle, neither stabilizer flips its main-study verdict. The most likely reason is that on low-entropy TinyStories text the logits never get anywhere near the instability these tools are built for. So over this range a bigger vocab just makes a bigger surface for z-loss to drag the model off the cross-entropy goal. The verdicts in Section 5.1 are not just an accident of the 8k point. The schedule and model-scale axes (sketched in the sweep script but not run here) are still the open question.

6 Discussion

A practitioner’s shortlist. If you can only port part of this stack, port Muon and QK-norm *together*. On their own they are not impressive but together they deliver. LayerScale and value residuals are a decent second tier. QK-gain and softcap can be dropped with no real cost at this scale. z-loss should just be dropped unless a longer schedule or a bigger vocab brings back the instability it is meant to handle.

vocab	params	z-loss Δ	softcap Δ	z-loss	softcap
8,192	34.1M	+0.0070	−0.0026	harmful	useful
16,384	42.5M	+0.0119	−0.0053	harmful	useful
32,768	59.3M	+0.0155	+0.0006	harmful	inert

Why z-loss flips sign. z-loss and softcap both exist to keep the logits from blowing up over long training. This study (short budget, small vocab, no broken runs) is exactly the case where that insurance is not needed. So it shows up as pure cost (z-loss) or pure no-op (softcap). The vocabulary sweep (Section 5.5) makes this sharper. Pushing the vocab from 8k to 32k is the direction where logit blow-up should get worse and the insurance should start to pay off. But it does not save z-loss. Its penalty just keeps getting more harmful and softcap only fades from mildly useful to nothing. On low-entropy text the softmax simply never reaches the unstable zone these tools target. The takeaway is that “stability” components should be tested against a baseline that is actually unstable and not just thrown into a recipe by default.

Scope and limitations.

- **Scale.** One model size (about 98M), one corpus (TinyStories, which is simple low-entropy text), one short budget (30.72M tokens). The *signs* of the big effects are probably solid. The inert/harmful calls on z-loss and softcap were the ones most likely to flip at a larger vocab. The vocabulary sweep in Section 5.5 tests this and finds they do not flip over the range I can reach, and z-loss actually gets worse all the way up to vocab 32k. The schedule and model-scale axes are still untested and they are the first thing to do next.
- **Fractional design.** I do not measure three-way or higher interactions. I do measure the main effects and the five targeted pairs. The strong Muon×QK-norm synergy means the single-factor numbers should be read as depending on context and not just added together.
- **Single quality metric.** Only the final validation loss, with no downstream-task or generation-quality test. Wall-clock cost is in Section 5.4 but it is on one GPU and one software stack. So the actual seconds depend on the setup even though the relative order does not.
- **Seeds.** Three seeds give a usable variance estimate. The per-factor std columns show the big effects sit well outside seed noise while QK-gain and softcap sit inside it.

7 Conclusion

The “modern transformer recipe” is not one solid thing. On a controlled TinyStories ablation almost all of its benefit is packed into **Muon plus QK-norm working together**. There is a smaller bit from LayerScale plus value residuals and then a trailing group of pieces that either do nothing (QK-gain, softcap) or hurt a little (z-loss) at this size. The single best setup is the full recipe with z-loss taken out. The bigger lesson is about method. Stack ingredients should earn their spot against matched multi-seed pairs and real interaction tests and not just by being piled on.

References

- [1] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, et al. Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24(240):1–113, 2023.

- [2] Ronen Eldan and Yuanzhi Li. Tinstories: How small can language models be and still speak coherent english? *arXiv preprint arXiv:2305.07759*, 2023.
- [3] Jonas Geiping and Tom Goldstein. Cramming: Training a language model on a single gpu in one day. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, pages 11117–11143, 2023.
- [4] Gemma Team, Google DeepMind. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*, 2024.
- [5] Alex Henry, Prudhvi Raj Dachapally, Shubham Shantaram Pawar, and Yuxuan Chen. Query-key normalization for transformers. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 4246–4253, 2020.
- [6] Pin-Lun Hsu, Yun Dai, Vignesh Kothapalli, Qingquan Song, Shao Tang, Siyu Zhu, Steven Shimizu, Shivam Sahni, Haowen Ning, and Yanning Chen. Liger kernel: Efficient triton kernels for llm training. *arXiv preprint arXiv:2410.10989*, 2024.
- [7] Keller Jordan, Yuchen Jin, Vlado Boza, Jiacheng You, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks. <https://kellerjordan.github.io/posts/muon/>, 2024.
- [8] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.
- [9] Sharan Narang, Hyung Won Chung, Yi Tay, Liam Fedus, Thibault Fevry, Michael Matena, Karishma Malkan, Noah Fiedel, Noam Shazeer, Zhenzhong Lan, Yanqi Zhou, Wei Li, Nan Ding, Jake Marcus, Adam Roberts, and Colin Raffel. Do transformer modifications transfer across implementations and applications? In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021.
- [10] Noam Shazeer. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.
- [11] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.
- [12] Hugo Touvron, Matthieu Cord, Alexandre Sablayrolles, Gabriel Synnaeve, and Hervé Jégou. Going deeper with image transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 32–42, 2021.
- [13] Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [14] Zhanchao Zhou, Tianyi Wu, Zhiyun Jiang, Fares Obeid, and Zhenzhong Lan. Value residual learning. *arXiv preprint arXiv:2410.17897*, 2024.

A Reproduction

```
python DataScripts/prepare_data.py           # one-time tokenization
python DataScripts/run_matrix.py --seeds 3    # 66 runs, resumable
python DataScripts/analyze_results.py         # tables + figures
python DataScripts/flip_sweep.py --axis vocab --auto-prepare # vocab sweep
python DataScripts/flip_analyze.py           # z-loss/softcap flip table
```

Per-run artifacts: `DataOutput/runs/<run_id>/{config.json,log.jsonl,summary.json}`. Aggregated tables and loss-curve figures: `DataOutput/analysis/{results.csv,main_effects.csv,interactions.csv,summary.md,loss_curves*.png}`. Vocab-sweep cells and their effects: `DataOutput/flip_runs/` and `DataOutput/analysis/flip_effects.csv`. All 66 main runs finished and 0 broke.